Linguaggi di programmazione

Francesco Lanzarini

1 Capitolo 1 - Macchine astratte, interpreti, compilatori

1.1 Che cosa è una macchina astratta? E in cosa si differenzia da una macchina fisica?

Una macchina astratta e' un insieme di strutture dati e algoritmi in grado di memorizzare e esegiure programmi, la parte esseziale di essa e' l'interprete. Una MA si differenza da una macchina fisica perche' e' capace, tramite alcune fasi, di eseguire programmi scritti in linguagggi diversi; infatti una macchina fisica e' in grado di eseguire programmi scritti nel suo linguaggio.

1.2 Che cos'è un interprete? In cosa consiste il ciclo fetch-decode-execute?

L'interprete e' il componente essenziale di una macchina astratta ed ha il compito di interpretare le istruzioni, esso e' costituito da: operazioni per elaborare istruzioni primitive, operazioni e strutture dati per controllare la sequenza di esecuzione, trasferimento dati e gestione memoria. Il ciclo fde consiste nel prelevare la giusta istruzione da eseguire (fetch), capire quale operzione e' necessario eseguire (decode) e eseguirla salvando eventualmente il risultato in una certa zona di memoria (execute).

1.3 Cos'è il linguaggio macchina?

Il linguaggio macchina e' il linguaggio che la macchina e' in grado di comprendere ed eseguire.

1.4 Possono esistere macchine diverse con lo stesso linguaggio macchina?

Si, perche' la stessa istruzione elementare puo' essere implementata in modo differente in due macchine diverse.

1.5 In quali modi è possibile implementare una macchina astratta? Elencare vantaggi e svantaggi delle varie tecniche.

Sono tre i principali modi per realizzare una macchina astratta:

1. Hardware: e' teoricamente sempre possibile ed ha la velocita' di esecuzione massima ma non e' per nulla flessbile per cui e' usata solo per macchine di basso livello o dedicate
2. Microprogrammi in ROM: rende microprogrammabile la macchina ospite, ha una maggiore falessibilita' ma minore velocita'.
3. Software di iterpretazione o simulazione: ovvero strutture dati e algoritmi della macchina astratta scritti in lunguaggio della macchina ospite, puo' essere implementata in qualsiasi macchina ospite, massima flessibilita' ma minima velocita'.

Nella pratica in realta' si usa una combinazione di queste tecniche.

1.6 Che cos'è un compilatore?

Il compilatore e' un programma che traduce un programma da una lingua L_1 ad una lingua L_0 preservandone la semantica.

1.7 Relativamente alla tecnica d'implementazione software, descrivere la tecnica d'implementazione interpretativa pura e quella compilativa pura.

Quando si esegue un'implementazione interpretativa pura si realizza un interprete scritto in L_0 (linguaggio macchina) per un linguaggio generico L, le singole istruzioni sono interpretate e poi eseguite. Mentre in un'implementazione compilativa pura i programmi scritti in L sono tradotti in programmi equivalenti in L_0 e poi eseguiti sulla macchina fisica

1.8 Quando un interprete si può dire corretto? Quando un compilatore si può dire corretto?

Un interprete si puo' dire corretto quando realizza una funzione parziale: $I_L^{L_0} : (P_r^L \times D)$ tale che $I_L^{L_0}(P_r^L, Input) = P^L(Input)$. Un compilatore si puo' dire corretto quando realizza una funzione totale: $C_{L,L_0} : Prog^L \to Prog_0^L$ tale che $C_{L,L_0}(P_r^L) = P_r c^{L_0}$

1.9 Confrontare l'implementazione di una macchina astratta su una macchina ospite per mezzo di un interprete o di un compilatore.

In un'implementazione interpretativa pura si ha una scarsa efficienza della macchina astratta perche' bisogna sommare ai tempi di esecuzione i tempi di decodifica del codice, inoltre nel caso una stessa istruzione dovesse ripetersi l'interprete deve decodificarla piu' volte; essa e' pero' un implementazione piu' flessibile, piu' facile da realizzare e non genera altro codice da memorizzare. Mentre un'implementazione comilativa pura puo' vantare un'ottima velocita' visto che tutto il codice e' gia' tradotto, essa e' pero' piu' difficile da realizzare inoltre e' poco flesssibile (bisognerebbe riscrivere il compilatore), astrae di piu' dal programma sorgente e occupa piu' memoria.

1.10 Come vengono implementate nella realtà le macchine astratte? Che cos'è la macchina intermedia?

Nella realta' e un misto delle due interpretazioni: di solito si compila il programma in una lingua intermedia L_i per una la macchina intermedia, suiccessivamente un interprete $I_{L_i}^{L_0}$ interpreta le istruzioni per l'esecuzione. Generalmente $I_{L_i}^{L_0}$ = interprete della macchina ospite + opportuni meccanismi chiamati supporto a run tim (SRT).

1.11 Quando si dice che una implementazione è di tipo interpretativo e quando di tipo compilativo? Fare esempi di linguaggi la cui implementazione è di un tipo o dell'altro.

Solitamente si dice che un implementazione e' interpretativa quando la maggior parte delle istruzioni vengono interpretate e poi eseguite, tipo in python e Java; mentre un'implementazione e' di tipo compilativo quando la maggior parte delle istruzioni sono prima tradotte e poi eseguite, tipo in C e C++.

1.12 Cos'è la Just-in-Time compilation?

La JIT e' una combinazione di AOT (ahead of time) compilation e interpretazione: si genera prima un codice intermedio (byte code in java) che viene poi eseguito passando per un compilatore on-demand man mano che esegue il programma tradotto in linguaggio macchina.

1.13 L'interprete e il compilatore si possono sempre realizzare?

Non sempre si possono realizzare, infatti il linguaggio di destinazione deve essere espressivo almeno quanto quello di partenza perchè se no alcune funzioni non sarebbero traducibili.

1.14 Che cosa è l'implementazione via kernel?

L'implementazione via kernel è un'implementazione che dato il linguaggio L prevede di individuare un sotto-insieme H, si scrive un compilatore in H che traduca da L a L_0 o un interprete in H per L e successivamente un compilatore/interprete per H. Questo fa si che si debbano scrivere due programmi ma essendo più vicini sia un lavoro più semplice e garantendo maggiore portabilità.

1.15 Cosa si intende per bootstrapping?

Per bootstrapping si intende una tecnica che consiste nell'utilizzare compilatori o interpreti per diverse lingue per tradurre un programma in un'altra lingua. Per esempio si consideri un compilatore che traduce da A a B e un altro da B a C, se volessimo tradurre un programma scritto in A nella lingua C possiamo usare i due compilatori al posto di crearne uno nuovo.

2 Capitolo 2 - Descrivere un linguaggio di programmazione

2.1 Quali sono i livelli di descrizione di un linguaggio?

I livelli di descrizione di un linguaggio sono quattro:

1. Sintassi: regole per cui una frase e' corretta.
2. Semantica: regole per cui una frase ha senso.
3. Pragmatica: il modo in cui sono usate le frasi corrette e sensate.
4. Implementazione: solo per i linguaggi eseguibili, come eseguire una frase rispettandone la semantica.

2.2 Sintassi: qual è l'aspetto lessicale e quale quello grammaticale di un linguaggio?

L'aspetto lessicale di un lunguaggio e' quello che specifica le parole che si possono usare, mentre quello grammaticale specifica come frasi corrette si possono ottenere a partire dal lessico.

2.3 Cos'è un alfabeto? Cos'è una parola o stringa? Cos'è A*? Tale insieme è enumerabile?

Un alfabeto e' un insieme tipicamente finito di simboli o caratteri, una parola o stringa e' una sequenza dinita di caratteri dell'alfabeto. Un linguaggio formale L su un alfabeto A esso e' un sottoinsieme di $A*$, $A*$ e' l'insieme di tutte le possibili parole sull'alfabeto A. $A*$ e' un infinito contabile, quindi e' contabile.

2.4 Definizione di potenza di una stringa. Definizione di potenza di un linguaggio. Definizione di chiusura/iterazione (o stella di Kleene) di un linguaggio.

- Potenza di una stringa: Data w una stringa, la sua potenza e' definita come: $w^0 = \epsilon$, $w^{n+1} = ww^n$
- Potenza di un linguaggio: Dato L un linguaggio, la sua potenza e' definita come: $L^0 = \{\epsilon\}$, $L^{n+1} = L \cdot L^n$
- Chiusura/iterazione (o stella di Kleene) di un linguaggio: $L* = Un \geq 0L^n$, $L^+ = Un \geq 1L^n$ (chiusura positiva)

2.5 Definizione di grammatica libera da contesto. Come si deriva una stringa? Qual è il linguaggio generato da una grammatica libera?

UN grammatca libera da contesto e' una quadrupla (NT,T,R,S) tale che:

- NT: e' l'insieme finito di non terminali.
- T: e' l'insieme finito di terminali.
- A $\in$ NT: e' il simbolo iniziale.
- R: e' l'insieme finito di produzioni della forma $V \to \alpha$

Per derivare una stringa si parte da P (non terminale, per esempio $P \to \epsilon|a|b|aPa|bPb$) e si applicano le produzioni. Il linguaggio generato da una grammatica libera e' l'insieme: $L(G) = \{w \in T|S \Rightarrow^* w\}$

2.6 Cos'è un albero di derivazione? Cos'è una derivazione canonica sinistra/destra (leftmost/rightmost)? Esiste una corrispondenza biunivoca tra alberi di derivazioni e derivazioni canoniche?

L'albero di derivazione e' un albero binario associato ad una particolare derivazione: esso e' ordinato e ogni nodo e' etichettato con un simbolo. Per derivazione canonica destra/sinistra si intende una derivazione che ad ogni passo va a riscrivere il non-terminale piu' a destra/sinistra. Esiste inoltre una corrispondenza biunivoca tra derivazioni leftmost/rightmost, piu' precisamente un albero di derivazione esiste una sola derivazione che lo genera.

2.7 Quando una grammatica è ambigua? (fare un esempio) Quando un linguaggio è ambiguo? (fare un esempio)

Una grammatica e' ambigua se esiste una stringa che ammette piu' alberi di derivazione, mentre un linguaggio ambiguo se tutte le grammatiche G tali che $L(G) = L$ sono ambigue.

2.8 È possibile rimuovere l'ambiguità dalla grammatica delle espressioni aritmetiche? Come?

Si, e' possibile introducendo una precedenza tra segni e l'associativita' degli stessi.

2.9 Cos'è l'albero di sintassi astratta? Che differenza c'è tra sintassi concreta e sintassi astratta? Cos'è lo zucchero sintattico?

L'albero di sintassi astratta e' un albero di derivazione da sintassi concreta senza zucchero sintattico. La sintassi concreta e' una sintassi piu' semplice ma ambigua, mentre la sintassi astratta e' piu' complessa ma non ambigua e con uso di zuccero sintattico, ovvero termini ausialiari usati per disanbiguare ma sintatticamente irrilevanti.

2.10 Fare esempi di vincoli sintattici contestuali. Possono essere catturati attraverso grammatiche libere?

Degli esempi di vincoli sintattici contestuali sono le dichiarazioi di variabili, type checking, corrispondenza tra dichiarazione e chiamata di funzione, visibilita' e scope, unicita' dei nomi e correttezza del flusso. Non possono essere catturati attraverso grammatiche libere.

2.11 *Cosa s'intende per semantica statica? E per semantica dinamica?*

Per semantica statica si intendono tutti i vincoli che possono essere verificati ispezionando il programma prima di mandarlo in esecuzione mentre quella dinamica considera il significato delle istruzioni eseguite.

2.12 Elencare le varie fasi in cui si articola un compilatore. Descrivere in dettaglio ogni singola fase.

2.13 A chi serve definire la semantica dinamica di un linguaggio e perché?

2.14 Quali tecniche si usano per dare semantica dinamica ad un linguaggio di programmazione?

2.15 Imparare le regole di semantica operazionale SOS per il semplice linguaggio presentato a lezione. Regole di valutazione: interna vs esterna, sinistra vs destra vs parallela.

2.16 Cosa s'intende per pragmatica? Cosa si intende per implementazione di un linguaggio?

3 Capitolo 3 - Analisi lessicale-Linguaggi regolari

3.1 Cosa fa l'analizzatore lessicale?

3.2 Cos'è un token?

3.3 Cos'è un pattern? Come lo si rappresenta? Cos'è un lessema?

3.4 Definizione di espressioni regolari (sintassi) e di linguaggio associato (semantica).

3.5 Quali sono i linguaggi regolari? I linguaggi finiti sono tutti regolari? Esistono linguaggi infiniti regolari?

3.6 Definizione di equivalenza tra espressioni regolari. Elencare alcune leggi di equivalenza.

3.7 Cos'è una definizione regolare e a cosa serve?

3.8 Definizione di NFA (automa finito nondeterministico). Discutere cosa si intende per nondeterminismo. Mettere in relazione la definizione formale con la rappresentazione grafica come diagramma di transizioni.

3.9 Come si definisce il linguaggio accettato da un NFA? Definizione di equivalenza tra NFA.

3.10 Definizione di DFA (automa finito deterministico). Discutere cosa si intende per determinismo. Mostrare che un DFA è un caso speciale di NFA, ovvero la classe dei DFA è un sottoinsieme della classe degli NFA.

3.11 Dato un NFA, come si ricava un equivalente DFA? Ovvero descrivere come è definita la costruzione per sottoinsiemi. Definizione di epsilon-closure e algoritmo associato. Qual è la complessità della costruzione per sottoinsiemi nel caso pessimo? Ovvero se NFA ha n stati, quanti stati può avere il DFA equivalente?

3.12 Dato i due punti precedenti, enunciare il teorema che dice che la classe dei linguaggi riconosciuto da NFA coincide con la classe dei linguaggi riconosciuti da DFA.

3.13 Come si costruisce un NFA a partire da una espressione regolare, in modo tale che il linguaggio riconosciuto dall'NFA sia lo stesso del linguaggio associato all'espressione regolare?

3.14 Definizione di grammatica regolare.

3.15 Come si associa ad una grammatica regolare un equivalente NFA?

3.16 Dato un DFA, come si costruisce una grammatica regolare equivalente?

3.17 Data una grammatica regolare, come si costruisce un'espressione regolare equivalente?

3.18 *Descrivere, con un diagramma riassuntivo, tutte le relazioni fra i formalismi introdotti: NFA, DFA, grammatiche regolari, espressioni regolari. Questo*